



AGENTIC ENGINEERING

AI4SE 2026



南京大学 邵栋

CONTENTS

01

三种编程模式

有哪几种写法?

02

模型能力拐点

为何现在才成立?

03

原型≠系统

能跑就够用了吗?

04

编程即理论

可信从何而来?

05

理解力负债

不懂有何代价?

06

可信性决策

何时该用哪种?

当前软件开发有三种模式共存

今天写代码，到底有哪几种方式？

- 手工编程、传统编程、人工编程、古法编程（人写+严谨）
- Vibe Coding 凭感觉编程、写意编程、氛围编程、沉浸式编程、（智能体写+低监督）
- Agentic Engineering 智能体工程（智能体写+高监督）：

Agentic Engineering

- Many people have tried to come up with a better name for this to differentiate it from vibe coding, personally my current favorite “agentic engineering”: 很多人试图为这种现象 (“用 LLM 智能体写代码”) 取一个更好的名字, 以区别于 “氛围编程”, 我个人目前最喜欢的是 “智能体工程” :

- "agentic" because the new default is that you are not writing the code directly 99% of the time, you are orchestrating agents who do and acting as oversight. - “智能体” 是因为现在 99%的时间里, 你不再直接编写代码, 而是在编排智能体并对其进行监督。
- "engineering" to emphasize that there is an art & science and expertise to it. It's something you can learn and become better at, with its own depth of a different kind.” - “工程” 是为了强调这其中艺术、科学和专业性。这是一门可以学习并不断提升的技能, 有着自己独特的深度。

- [Andrej Karpathy 2026.02](#)

- [Co-Founder, OpenAI](#)



2025-11 能力拐点

为什么「智能体工程」现在才成立？

「在我的感受中，十一月(2025年)发布的 GPT-5.2 和 Opus 4.5 确实代表了一个拐点——就是模型的增量进步突然越过了一条看不见的能力线。一下子，一大批更难的编程问题变得可以解决了。」

—— **Simon Willison**

若模型能力不达标， 则 Agentic Engineering 不成立。

可信性

代码「能跑」，为什么不等于「能用」？



“能跑≠可信”：软件可信性问的不是“代码现在跑没跑通”，而是你有没有充分理由相信：换个时间、换种情况，它照样不出错。

原型，不等于系统



例子：王主任（25 年急诊经验）用 Claude Code 搭出了 89% 准确率的分诊原型——“差不多能用了”。但距真正用在急诊室，还差一整座冰山：

容错降级 · 系统崩了怎么办 **审计合规** · 纠纷谁说得清
并发实时 · 百人同时用 **模型监控** · AI 变笨谁知道
数据安全 · 隐私谁来守 **系统集成** · 新设备接得上

医生能定义分诊逻辑，但让系统 7×24 可靠运行的工程，超出其专业。AI 赋能的软件工程师，价值就在构建这座水下冰山。

编程的产物不是程序，而是“理论”

可信，到底从哪里来？

编程的根本目的不是产出程序，而是让程序员在头脑中建立「问题如何被程序求解」的理论。

Naur 1985 · Programming as Theory Building | 理论是一种能力、而非命题集合 (Ryle) —— 持有理论的人能回答关于程序的「任何问题」。

核心命题：源代码只是这套理论「写下来的有损投影」

代码说得清

「这段代码做了什么」

代码说不清

「为什么这么写、当时考虑过哪些备选、哪些边界被有意排除」

「有损」在哪：理论中关于「不存在之物」的知识，写下来的产物里没有位置

被排除的方案

被假定不会发生的输入

被刻意保持的不变量

如何检验一个人是否持有“理论”

- Naur 给出三条具体的判据；三条合起来指向同一个结论——代码、文档、测试加起来也无法完整承载理论。
- 1、**双向映射**：能把程序的每一个部分对应到所要解决的现实世界事务，在代码与问题域之间建立稳定的双向映射，而不仅是知道某段代码的输入输出。
- 2、**设计辩护**：能为每一项设计选择给出辩护，尤其当被问“为什么不那样写”，不仅知道当前的解，还知道当前解所处的可能性空间：哪些路径被探索过、为什么被排除。
- 3、**有原则的扩展与修改**：能对程序作有原则的扩展，“有原则”指新代码与原有代码共享同一套不变量、同一套问题域抽象，而不是在缝里塞东西。

"程序已死" (the program is dead)

失去理论的系统仍能运行，**却不能再被有意识地演化**：

bug 修复反复破坏看似无关的功能，
重构不断撞见"**这段为什么在？没人记得**"的化石。

传统软件开发，当所有原始团队人员离开后，程序死亡。

程序死亡会有什么问题？

程序仍然能运行，但：



生产事故根因分析

出事时找不到

出了问题，没有人讲得清某个关键模块为什么这么写，找不到问题，也解决不了问题。



需求变更

要改时改不动

想在原有设计上扩展，却没人说得清当初的约束与取舍，不敢改也改不动，最终要么推倒重来，要么埋下新的隐患。



AI 也读不懂

Agent 只能靠猜

没有被记录下来的设计意图，AI / Agent 接手时只能靠猜，给出看似合理、实则破坏隐含约束的改动。

对中大型、长生命周期的真实项目，这是无法承受之重。

为什么人仍然需要理解代码（重建理论）

- 信任的根本来源不是代码的行为正确性，而是有人持有关于“代码为什么这样工作”的理论。模型再强、基准再高，只要没人持有理论，“代码能跑”就不等于“系统可信”。
- 当智能体写下大量代码、而无人持有其背后的理论时，等价于Naur描述的“团队全部离开”，只是这次的理论缺位，是因为代码从一开始就没谁持有理论。AI 生成的代码一开始就死亡了！

理解力负债

没人真懂代码，会付出什么代价？

- Addy Osmani 2026 年 3 月：**comprehension debt理解力负债** = 系统中存在的代码量与"任何人真正理解的代码量"之间不断扩大的差距。
- 无声累积，滋生虚假信心：代码看起来干净、风格一致、测试甚至能通过，却无人理解其边界条件与隐含假设。事件触发瞬间 一次性兑付。
- 对比技术债（Cunningham 1992）：有意识地选择的次优设计，未来"还本付息"。它会通过摩擦自报告：构建变慢、依赖纠缠、修改某模块时的预感性恐惧。

偿还理解力负债

1

代码评审作理论传递

评审本有两个功能——质量门禁 + 知识分发；在 AI 生成速度下，光挑语法、找缺陷已经不够。评审者真正的活，是从「找 bug」转向「理论传递」：逼出代码背后的「为什么」，把隐含假设与设计取舍问出来，确保至少有一个人能脱离 AI 提示讲清这段逻辑。

Faros AI 《AI Engineering Report 2026》（遥测 2.2 万名开发者）：高 AI 采用下，PR 评审中位时长 +441% —— 理论传递的真实工程成本，第一次被看见。

2

显式规格与章程文件 (spec & constitution)

把「为什么这么做」写入规格、ADR (Architecture Decision Record, 架构决策记录)、章程文件 (constitution)，让智能体工作前先读这份理论。这一步把团队头脑中的心智模型 (mental model) 显式外化 (externalize) 为可被智能体与未来工程师共同消费的工程产物。**但要清醒：**文字终究有损，写下来是必要而非充分，替代不了鲜活的理论。

3

让理解力负债「可见」——先测量，再偿还

理解力负债看不见，所以第一步不是还、而是先找：把它定位出来，再像技术债一样排优先级偿还。两个抓手——一是定期做 system walk：让人沿整条链路追一个请求、逐层讲清「为什么这么写」，谁讲不清，那段就是负债热点；二是在 PR 里加一条探针——「不看 AI 提示，你能讲清这段代码吗？」答不上来，就是正在新增负债。

“AI 写代码人需不需要懂？”

什么时候必须有人懂、该用哪种模式？

- **行为正确性**：给定输入产生预期输出。测试 / 类型 / 形式化可保证。AI 能独立达标。
- **因果可解释性**：出错时能否在合理时间定位根因。靠工程师对代码的理解，不是测试。
- **变更可控性**：修改 A 时不会破坏 B、C、D。依赖“理论持有”，AI 不持有领域上下文，尤其是隐性知识。
- **责任可归属**：出事谁负责、能否合规审计。制度层面，结构上必然要求人在回路。

可信性预算决策表

工作模式选择不是个人偏好，而是任务画像的函数。

任务类型	行为正确	因果可解释	变更可控	责任可归属	推荐工作模式
一次性工具 / 探索 demo	低	—	—	—	Vibe Coding
长期维护的内部工具	中	中	中	低	Agentic (轻量)
面向用户的生产功能	高	高	高	中	Agentic (标准)
安全关键 / 合规代码	极高	极高	极高	极高	Agentic + 手工核心
核心抽象 (OS/DB/...)	极高	极高	极高	极高	手工编程 (1%)

读法： 同一名工程师在同一天里，会在表格不同行之间切换。



THANKS